Review dan Komentar Sumber Belajar Neural Network

Pembelajaran PyTorch, ANN untuk PV, dan Peramalan Deret Waktu

Disusun Marsa Naufal (04231047)

20 Oktober 2026

Ringkasan

- **learnpytorch.io**: sumber belajar terbaik di antara keempatnya untuk pemula, tetapi bukan sumber untuk data finansial/energi.
- **Neural-Network-for-PV-system**: notebook yang berjalan, namun memuat kebocoran skala data (scaling) dan arsitektur yang janggal; jangan ditiru apa adanya.
- **TimeSeriesForecastingDairyProduction**: analisis deret waktu klasik (R), tanpa validasi; berguna sebagai contoh dekomposisi, bukan sebagai contoh evaluasi.
- **lstm-load-forecasting**: rancangan eksperimen paling rapi (grup fitur, pembanding statistik, rolling forecast), tetapi lingkungannya sudah usang.

1 learnpytorch.io (Zero to Mastery, Daniel Bourke)

Isi. Kursus gratis berbentuk buku daring dan video: 00 Fundamentals, 01 Workflow, 02 Neural Network Classification, 03 Computer Vision, 04 Custom Datasets, 05 Going Modular, 06 Transfer Learning, dan tiga proyek milestone (experiment tracking, mereplikasi paper, deployment).

Kelebihan	**Kekurangan**
Gratis, kode-dulu: tiap konsep langsung dijalankan di Colab.	Tidak ada materi tabular/finansial atau deret waktu; contohnya gambar dan data mainan.
Alur kerja konsisten (data → model → loss → loop → evaluasi) yang mudah dihafal.	Terakhir diperbarui sekitar April 2023; bagian PyTorch 2.0 belum lengkap pada saat halaman dibaca.
Bab 02 memperkenalkan non-linearitas (ReLU) dan `BCEWithLogitsLoss` dengan benar.	Bergantung pada Google Colab dan mengasumsikan pengetahuan Python/ML dasar; halaman merekomendasikan bootcamp berbayar penulisnya.

Komentar. Cocok sebagai fondasi, dan pola di notebook `finance_nn.ipynb` (MLP, `BCEWithLogitsLoss`, mini-batch, Adam) mengikuti bab 02. Kekurangan terbesar bagi proyek keuangan atau energi: tidak ada pembahasan **split berurutan waktu**, kelas tidak seimbang, atau pemilihan hyperparameter dengan validasi. Bagian itu harus ditambah dari sumber lain.

2 Neural-Network-for-PV-system (beki2022)

Isi (dari notebook yang saya baca). Regresi tabular: 6 fitur iradiasi (`Ir11, Ir21, Ir31, Ir12, Ir22, Ir32`) memprediksi target `Voltage_pmax`; 36.071 baris, dimuat dari Google Drive. Pembagian acak 80/20 (28.856 latih, 7.215 uji), `MinMaxScaler`, jaringan Keras 9 lapisan Dense, Adam (lr 10^{-4}), 1000 epoch, batch 10, `validation_split=0.1`.

Kelebihan	Kekurangan
Masalahnya jelas dan relevan (prediksi titik daya maksimum PV dari iradiasi).	Data tidak ikut repositori (path `/content/drive/MyDrive/...`); tidak dapat dijalankan ulang oleh orang lain.
Kurva loss latih vs validasi diplot; ada MAE selain MSE.	Nilai metrik akhir tidak tersimpan di keluaran notebook (sel pencetak MAE/RMSE tidak punya keluaran), jadi klaim performa tidak dapat diperiksa.
Model disimpan ke `.h5` sehingga tidak perlu dilatih ulang.	Tidak ada `random_state` pada split sehingga hasil berubah tiap kali dijalankan.

Masalah metodologi yang saya temukan

1. **Scaler di-fit ulang pada data uji.** Sel 14 memakai `fit_transform` pada `x_train`, lalu sel 15 memakai `fit_transform` lagi pada `x_test`. Seharusnya `transform` saja dengan scaler dari data latih. Akibatnya skala data uji berbeda dari data latih dan metrik uji tidak valid.

2. **Arsitektur janggal.** Ada lapisan `softmax` 84 unit di tengah jaringan (softmax untuk output klasifikasi, bukan lapisan tersembunyi regresi), dua lapisan `sigmoid` di tengah, dan dua lapisan `linear` berurutan yang secara matematis dapat digabung menjadi satu. Tidak ada justifikasi; model sederhana (2–3 lapisan ReLU) hampir pasti cukup untuk 6 fitur.

3. **Overfitting dan tidak ada early stopping.** Pada epoch 1000, loss latih $\approx$ 4,9 sedangkan loss validasi $\approx$ 21,6 (MAE validasi $\approx$ 1,48). Selisih ini menandakan jaringan menghafal; 1000 epoch dengan batch 10 (2.597 langkah per epoch) tanpa `EarlyStopping` memboroskan waktu.

4. **Pelatihan ganda tanpa dicatat.** Sel 20 melatih lagi 50 epoch pada *seluruh* data latih (termasuk bagian validasi) bila berkas `.h5` belum ada, lalu menyimpan model. Model yang dievaluasi bukan model yang kurvanya diplot.

5. **Plot prediksi tidak sejajar.** `y_test` diplot dengan indeks asli (acak karena split acak), sedangkan prediksi diplot pada `range(len(y_train)+50, …)`. Dua garis itu tidak berada pada sumbu yang sama, sehingga grafik tidak dapat dibaca sebagai perbandingan.

6. **Pencampuran `keras` dan `tf.keras`** (impor ganda) berisiko menimbulkan konflik versi (`keras` 2.8.0 terdeteksi).

Saran. Pisahkan data latih/validasi/uji, `fit` scaler hanya pada latih, pakai `EarlyStopping`, sederhanakan model, simpan metrik ke keluaran, dan sertakan contoh data kecil di repositori.

3 TimeSeriesForecastingDairyProduction (themodernturing)

Isi (dari README). Analisis R atas produksi susu dan es krim bulanan di California: dekomposisi (rata-rata bergerak dan STL), ARIMA pada komponen sisa, auto-ARIMA dari paket `forecast`, dan ramalan 12 bulan dengan interval kepercayaan.

Kelebihan	Kekurangan
Alur klasik yang benar secara konsep: plot, ACF/PACF, dekomposisi, model, ramalan.	Tidak ada validasi: tidak ada data uji, tidak ada RMSE/MAE/MAPE, jadi kualitas ramalan tidak dapat dinilai.
Dekomposisi STL membantu memahami tren dan musiman.	Data `cadairydata.csv` tidak ada di repositori; sumber dan rentang waktu tidak dijelaskan.
Ramalan menyertakan interval kepercayaan.	Metode akhir ambigu (ARIMA pada sisa vs auto-ARIMA); nama berkas di README tidak sama dengan berkas sebenarnya; bagian lisensi kosong.

Komentar. Bukan contoh yang baik untuk neural network (tidak ada model neural sama sekali), tetapi berguna sebagai pembanding *baseline* statistik. Untuk data bulanan, model musiman (SARIMA/ETS) dan evaluasi dengan hold-out 12 bulan adalah langkah minimal yang absen.

4 lstm-load-forecasting (dafrie)

Isi. Proyek demo peramalan beban listrik Swiss dengan LSTM (Keras 2.0.2/TensorFlow), data per jam dari ENTSO-E dan cuaca dari Dark Sky, ukuran total 20756×78. Enam grup fitur (hanya ENTSO-E, hanya kalender, hanya cuaca, dan kombinasinya) dibandingkan terhadap pembanding TBATS dan ARIMA (R), plus rolling forecast. Repositori ini bertanda 210 bintang dan lisensi MIT.

Kelebihan	Kekurangan
Desain eksperimen paling rapi: grup fitur dipisah sehingga kontribusi tiap grup dapat dibandingkan.	Dependensi sangat tua (Keras 2.0.2); hampir pasti tidak berjalan di lingkungan modern tanpa penyesuaian.
Pembanding statistik (TBATS, ARIMA) dan *rolling forecast* merupakan evaluasi yang realistis.	Butuh API key (`config.json`); layanan Dark Sky sudah tidak tersedia sehingga data cuaca tidak dapat diambil ulang.
Struktur berkas dan notebook per model jelas; model tersimpan (HDF5).	README tidak menyebut metrik, ruang pencarian hyperparameter, atau angka hasil utama.

Komentar. Ini sumber paling dekat dengan kebutuhan deret waktu serius dan paling layak ditiru dari sisi desain: bandingkan model neural dengan baseline statistik, ablasi fitur, dan evaluasi bergulir. Saya **tidak memverifikasi** angka hasilnya; README tidak memuatnya dan notebook tidak saya jalankan.

5 Perbandingan ringkas

	learnpytorch	PV notebook	Dairy (R)	LSTM load
Tujuan	Belajar PyTorch	Regresi PV	Ramal bulanan	Ramal beban
Reproducible	Ya (Colab)	Tidak (data Drive)	Sebagian (CSV hilang)	Sebagian (API tua)
Validasi benar	Tidak dibahas	Bermasalah	Tidak ada	Ya (rolling)
Baseline pembanding	—	Tidak ada	Tidak ada	Ya (TBATS, ARIMA)
Layak ditiru	Fondasi	Tidak apa adanya	Sebagian	Desainnya

6 Catatan terkait lampiran

6.1 Laporan akhir: ANN vs P&O untuk MPPT

Ini bukan website yang diminta, tetapi karena terlampir saya catat temuan yang perlu diperbaiki:

- **Kontradiksi angka.** Tabel 7.4 menyebut efisiensi P&O 84,26 % dan ANN 87,9 %, tetapi Bab 8.1 menulis "P&O 87,9 % sedangkan ANN 84,26 %". Salah satu harus dikoreksi; tabel yang benar menurut seluruh bab 7.
- **Satuan temperatur.** Bab 3.3 menulis tegangan sel turun "2,2 V" per derajat. Nilai yang lazim adalah sekitar 2,2 mV per °C per sel.
- **Persamaan 6.6.1** memakai konstanta $\frac{1}{12}$ sementara teks menyebut N pola latih; seharusnya $\frac{1}{N}$.
- **Data latih berasal dari P&O.** ANN meniru P&O (200.000 sampel), sehingga batas kualitasnya adalah kualitas P&O sendiri; ini perlu dinyatakan sebagai keterbatasan. Judul "Levenberg–Marquardt" di 6.6 dan teks backpropagation di Bab 5.6 tidak diberi hubungan yang jelas.
- **Splitting.** Pembagian latih/validasi/uji pada bab 7.2(I) ($R \approx 0{,}989$ di semua subset) perlu keterangan bahwa data berasal dari satu simulasi yang sama, sehingga R tinggi tidak membuktikan generalisasi ke kondisi cuaca lain.

6.2 Tangkapan layar TensorFlow Playground

Pengaturan: tugas *regression*, aktivasi tanh, learning rate 0,03, batch 10, 2 lapisan tersembunyi (4 dan 5 neuron), L1 dengan rate 0, noise 0, epoch 1.023, test loss dan training loss 0,000. Loss ≈ 0 wajar karena noise 0 dan pola data berupa gradien hampir linear; jaringan dua lapisan berlebihan untuk pola ini (jaringan yang jauh lebih kecil sudah cukup). Playground cocok untuk intuisi tentang kapasitas dan learning rate, tetapi hasilnya tidak berlaku untuk data nyata yang bernoise dan memiliki fitur kategorikal.

7 Rekomendasi

1. Untuk belajar, ikuti learnpytorch.io bab 00–02, lalu terapkan pada data sendiri (sudah dilakukan di `NN/finance_nn.ipynb`).

2. Untuk deret waktu, tiru desain lstm-load-forecasting (baseline statistik, ablasi fitur, rolling forecast), tetapi tulis ulang dengan TensorFlow/PyTorch modern.

3. Jangan menyalin notebook PV apa adanya; perbaiki scaler, arsitektur, early stopping, dan plot lebih dulu.

4. Selalu pisahkan data uji sebelum melakukan apa pun, dan untuk deret waktu pisahkan secara kronologis, bukan acak.